Packages and JAVA Library:

Introduction: Packages are used in Java in order to prevent naming conflicts, control access, make searching/locating and usage of classes, interfaces, enumerations, and annotations easier, etc.

A Java package can be defined as a grouping of related types (classes, interfaces, enumerations, and annotations) providing access protection and namespace management.

Java packages are of two types:

- Built-in Java Packages
- User-defined Java Packages

Defining Package

While creating a package, you should choose a name for the package and include a package statement along with that name at the top of every source file that contains the classes, interfaces, enumerations, and annotation types that you want to include in the package.

The package statement should be the first line in the source file. There can be only one package statement in each source file, and it applies to all types in the file.

If a package statement is not used then the class, interfaces, enumerations, and annotation types will be placed in the current default package.

To compile the Java programs with package statements, you have to use -d option as shown below.

javac -d Destination_folder file_name.java

Then a folder with the given package name is created in the specified destination, and the compiled class files will be placed in that folder.

Example

package pack1;

```
public class Employee
{
public String name;
public void display()
{
System.out.println("Name of the employee is "+name);
}
}
```

Compiling: >javac –d . Employee.java

Importing the package pack1:

```
import pack1.Employee;
class PackageDriver
{
public static void main(String args[])
{
Employee emp=new Employee();
emp.name="ABC";
emp.display();
}
}
```

Path and ClassPath:

The **ClassPath** is a parameter in the Java Virtual Machine(JVM) or the Java compiler that is used by a system or application ClassLoader to locate and load compiled Java bytecodes stored in the “.class” file. On the other hand, The **Path** is also an environment variable path that behaves as a mediator between the operating system and developer to inform binary file path.

Path

PATH is an environment variable that is used to find and locate binary files like “java” and “javac” and to locate needed executables from the command line or Terminal window. To set the path, we’re supposed to include or mention JDK_HOME/bin directory in a PATH environment variable. The PATH can not be overridden by providing command and PATH is only used by the operation system(OS) to find binary files.

Syntax

```
// To set PATH in the window OS.
set PATH=%PATH%;C:\Program Files\Java\JDK1.5.10\bin


// To set PATH in Unix OS
export PATH=${PATH}:/opt/Java/JDK1.5.10/bin
```

Classpath

Classpath is an environment variable that is used by the application ClassLoader or system to locate and load the compiled Java bytecodes stored in the .class file. To set CLASSPATH. the CLASSPATH can be overridden by adding classpath in the manifest file and by using a command like set -classpath. the CLASSPATH is only used by Java ClassLoaders to load class files.

Syntax

```
// To set CLASSPATH in window OS.
set CLASSPATH=%CLASSPATH%;C:\Program Files\Java\JDK1.5.10\lib


// To set PATH in Unix OS.
export CLASSPATH=${CLASSPATH}:/opt/Java/JDK1.5.10/lib
```

java.lang Package and its Classes

Java.lang package contains the classes that are fundamental to the design of the Java programming language. Wrapper Classes like Integer, Boolean, Char etc are part of lang package. We're not supposed to import any class of java.lang package explicitly as being included by default.

some of the categories of classes of java.lang package.

- **Wrapper Classes** − java.lang package contains wrapper classes like Integer, Boolean, Float, Double etc for primitive values. These wrapper classes provides many useful methods and can be used as objects where primitive value is not to be used.
- **Mathematical operations** − Math and StrictMath classes are rich with mathematical operations.
- **Thread operations** −Thread, ThreadLocal, ThreadGroup etc. are important classes for multi-threading programs.

Following is the list of some of the important classes in java.lang.package:

- Boolean − The Boolean class wraps a value of the primitive type boolean in an object. An object of type Boolean contains a single field whose type is boolean.
- Byte − The Byte class class wraps a value of primitive type byte in an object. An object of type Byte contains a single field whose type is byte.
- Character − The Character class offers a number of useful class (i.e., static) methods for manipulating characters.
- Double − The Double class wraps a value of primitive type double in an object. An object of type Double contains a single field whose type is double.
- Float − The Float class wraps a value of primitive type float in an object. An object of type Float contains a single field whose type is float.
- Integer − The Integer class wraps a value of primitive type int in an object. An object of type Integer contains a single field whose type is int.
- Long − The Long class wraps a value of the primitive type long in an object. An object of type Long contains a single field whose type is long.
- Math − The Math class contains methods for performing basic numeric operations such as the elementary exponential, logarithm, square root, and trigonometric functions.
- Short − The Short class wraps a value of primitive type short in an object. An object of type Short contains a single field whose type is short.
- String − Strings, which are widely used in Java programming, are a sequence of characters. In Java programming language, strings are treated as objects. The Java platform provides the String class to create and manipulate strings.
- StringBuffer − The StringBuffer class is mutable sequence of characters. StringBuffer can be used to modify the content of a String with ease. It provides many utility functions to manipulate a string.
- StringBuilder − The StringBuilder class is mutable sequence of characters. This provides an API compatible with StringBuffer, but with no guarantee of synchronization.
- Thread − The Thread class is a thread of execution in a program. The Java Virtual Machine allows an application to have multiple threads of execution running concurrently.

class Math:

The **java.lang.Math** class contains methods for performing basic numeric operations such as the elementary exponential, logarithm, square root, and trigonometric functions.

Syntax:

public final class Math extends Object

Methods

Sr.No.	Method & Description
1	**static double abs(double a)** This method returns the absolute value of a double value.
2	**static float abs(float a)** This method returns the absolute value of a float value.
3	**static int abs(int a)** This method returns the absolute value of an int value.
4	**static long abs(long a)** This method returns the absolute value of a long value.
5	**static double acos(double a)** This method returns the arc cosine of a value; the returned angle is in the range 0.0 through pi.
6	**static double asin(double a)** This method returns the arc sine of a value; the returned angle is in the range -pi/2 through pi/2.
7	**static double atan(double a)** This method returns the arc tangent of a value; the returned angle is in the range -pi/2 through pi/2.
8	**static double cbrt(double a)** This method returns the cube root of a double value.
9	**static double ceil(double a)** This method returns the smallest (closest to negative infinity) double value that is greater than or equal to the argument and is equal to a mathematical integer.
10	**static double cos(double a)** This method returns the trigonometric cosine of an angle.

11 **static double cosh(double x)**
This method returns the hyperbolic cosine of a double value.

12 **static double exp(double a)**
This method returns Euler's number e raised to the power of a double value.

13 **static double floor(double a)**
This method returns the largest (closest to positive infinity) double value that is less than or equal to the argument and is equal to a mathematical integer.

14 **static double log(double a)**
This method returns the natural logarithm (base e) of a double value.

15 **static double log10(double a)**
This method returns the base 10 logarithm of a double value.

16 **static double max(double a, double b)**
This method returns the greater of two double values.

17 **static float max(float a, float b)**
This method returns the greater of two float values.

18 **static int max(int a, int b)**
This method returns the greater of two int values.

19 **static long max(long a, long b)**
This method returns the greater of two long values.

20 **static double min(double a, double b)**
This method returns the smaller of two double values.

21 **static float min(float a, float b)**
This method returns the smaller of two float values.

22 **static int min(int a, int b)**
This method returns the smaller of two int values.

23 **static long min(long a, long b)**
This method returns the smaller of two long values.

24 **static double pow(double a, double b)**
This method returns the value of the first argument raised to the power of the second argument.

25 **static double random()**
This method returns a double value with a positive sign, greater than or equal to 0.0 and less than 1.0.

26 **static double rint(double a)**
This method returns the double value that is closest in value to the argument and is

equal to a mathematical integer.

27 **static long round(double a)**
This method returns the closest long to the argument.

28 **static int round(float a)**
This method returns the closest int to the argument.

29 **static double sin(double a)**
This method returns the hyperbolic sine of a double value.

30 **static double sinh(double x)**
This method Returns the hyperbolic sine of a double value.

31 **static double sqrt(double a)**
This method returns the correctly rounded positive square root of a double value.

32 **static double tan(double a)**
This method returns the trigonometric tangent of an angle.r

33 **static double tanh(double x)**
This method returns the hyperbolic tangent of a double value.

34 **static double toDegrees(double angrad)**
This method converts an angle measured in radians to an approximately equivalent angle measured in degrees.

35 **static double toRadians(double angdeg)**
This method converts an angle measured in degrees to an approximately equivalent angle measured in radians.

The **wrapper class in Java** provides the mechanism *to convert primitive into object and object into primitive*.

Since J2SE 5.0, **autoboxing** and **unboxing** feature convert primitives into objects and objects into primitives automatically. The automatic conversion of primitive into an object is known as autoboxing and vice-versa unboxing.

Java is an object-oriented programming language, so we need to deal with objects many times like in Collections, Serialization, Synchronization, etc. Let us see the different scenarios, where we need to use the wrapper classes.

- **Change the value in Method:** Java supports only call by value. So, if we pass a primitive value, it will not change the original value. But, if we convert the primitive value in an object, it will change the original value.
- **Serialization:** We need to convert the objects into streams to perform the serialization. If we have a primitive value, we can convert it in objects through the wrapper classes.
- **Synchronization:** Java synchronization works with objects in Multithreading.
- **java.util package:** The java.util package provides the utility classes to deal with objects.
- **Collection Framework:** Java collection framework works with objects only. All classes of the collection framework (ArrayList, LinkedList, Vector, HashSet, LinkedHashSet, TreeSet, PriorityQueue, ArrayDeque, etc.) deal with objects only.

The eight classes of the *java.lang* package are known as wrapper classes in Java. The list of eight wrapper classes are given below:

Primitive Type	Wrapper class
boolean	Boolean
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double

Autoboxing

The automatic conversion of primitive data type into its corresponding wrapper class is known as autoboxing, for example, byte to Byte, char to Character, int to Integer, long to Long, float to Float, boolean to Boolean, double to Double, and short to Short.

Since Java 5, we do not need to use the valueOf() method of wrapper classes to convert the primitive into objects.

```
public class WrapperExample1{
public static void main(String args[]){
//Converting int into Integer
int a=20;
Integer i=Integer.valueOf(a);//converting int into Integer explicitly
Integer j=a;//autoboxing, now compiler will write Integer.valueOf(a) internally

System.out.println(a+" "+i+" "+j);
}}
```

Unboxing

The automatic conversion of wrapper type into its corresponding primitive type is known as unboxing. It is the reverse process of autoboxing. Since Java 5, we do not need to use the intValue() method of wrapper classes to convert the wrapper type into primitives.

Wrapper class Example: Wrapper to Primitive

```
public class WrapperExample2{
public static void main(String args[]){
//Converting Integer to int
Integer a=new Integer(3);
int i=a.intValue();//converting Integer to int explicitly
int j=a;//unboxing, now compiler will write a.intValue() internally

System.out.println(a+" "+i+" "+j);
}}
```

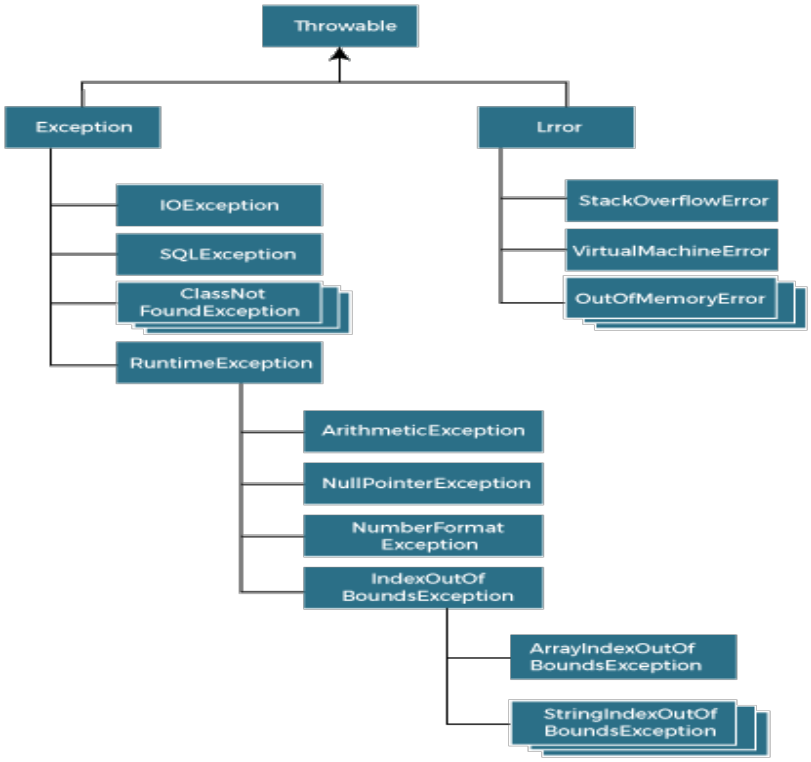
Exception Handling

Introduction:
The **Exception Handling in Java** is one of the powerful *mechanism to handle the runtime errors* so that the normal flow of the application can be maintained.

In Java, an exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions. These exceptions can occur for various reasons, such as invalid user input, file not found, or division by zero. When an exception occurs, it is typically represented by an object of a subclass of the java.lang.Exception class.

Hierarchy of Java Exception classes

The java.lang.Throwable class is the root class of Java Exception hierarchy inherited by two subclasses: Exception and Error. The hierarchy of Java Exception classes is given below:



Keywords throws and throw, try, catch, and finally Blocks:

Java provides five keywords that are used to handle the exception. The following table describes each.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

The try-catch Block

One of the primary mechanisms for handling exceptions in Java is the try-catch block. The try block contains the code that may throw an exception, and the catch block is used to handle the exception if it occurs. Here's a basic example:

```
try {
    // Code that may throw an exception
} catch (ExceptionType e) {
    // Exception handling code
```

```
}
```

Handling Multiple Exceptions

You can handle multiple types of exceptions by providing multiple catch blocks, each catching a different type of exception. This allows you to tailor your exception handling logic based on the specific type of exception thrown. Here's an example:

```
try {
   // Code that may throw an exception
} catch (IOException e) {
   // Handle IOException
} catch (NumberFormatException e) {
   // Handle NumberFormatException
} catch (Exception e) {
   // Handle any other exceptions
}
```

```
public class TryCatchExample {

   public static void main(String[] args) {
      try
      {
      int data=50/0; //may throw exception
      }
         //handling the exception
      catch(ArithmeticException e)
      {
         System.out.println(e);
      }
      System.out.println("rest of the code");
   }

}
```

Multi-catch block

```
public class MultipleCatchBlock {

   public static void main(String[] args) {

        try{
            int a[]=new int[5];
            a[5]=30/0;
           }
           catch(ArithmeticException e)
              {
               System.out.println("Arithmetic Exception occurs");
              }
           catch(ArrayIndexOutOfBoundsException e)
              {
               System.out.println("ArrayIndexOutOfBounds Exception occurs");
              }
           catch(Exception e)
              {
               System.out.println("Parent Exception occurs");
              }
           System.out.println("rest of the code");
   }
}
```

The finally Block

In addition to try and catch, Java also provides a finally block, which allows you to execute cleanup code, such as closing resources, regardless of whether an exception occurs or not. The finally block is typically used to release resources that were acquired in the try block. Here's an example:

```
try {
   // Code that may throw an exception
} catch (Exception e) {
   // Exception handling code
} finally {
   // Cleanup code
}
```

```
class FinallyDemo{
public static void main(String args[])
{
try{
throw new ArithmeticException();
}catch(NullPointerException e)
{
System.out.println("Catch Block executed");
}
finally{
System.out.println("Finally Block executed");
}
}
}
```

throw keyword

```
public class TestThrow1 {
   //function to check if person is eligible to vote or not
   public static void validate(int age) {
      if(age<18) {
         //throw Arithmetic exception if not eligible to vote
         throw new ArithmeticException("Person is not eligible to vote");
      }
      else {
         System.out.println("Person is eligible to vote!!");
      }
   }
   //main method
   public static void main(String args[]){
      //calling the function
      validate(13);
      System.out.println("rest of the code...");
  }
}
```

Throws keyword

```
import java.io.*;
```

```
class ThrowsDemo
{
void display() throws IOException
{
throw new IOException();
}
}
public class ThrowsDriver
{
static public void main(String args[])throws IOException
{
ThrowsDemo a=new ThrowsDemo();
a.display();
System.out.println("Rest of the code");
}
}
```

Class Throwable

The **Java Throwable** class is the superclass of all errors and exceptions in the Java language. Only objects that are instances of this class (or one of its subclasses) are thrown by the Java Virtual Machine or can be thrown by the Java throw statement.

Syntax:
public class Throwable extends Object implements Serializable

methods

Sr.No.	Method & Description
1	**Throwable fillInStackTrace()** This method fills in the execution stack trace.
2	**String getMessage()** This method returns the detail message string of this throwable.
3	**StackTraceElement[] getStackTrace()** This method provides programmatic access to the stack trace information printed by printStackTrace().
4	**void printStackTrace()** This method prints this throwable and its backtrace to the standard error stream.

5	**void printStackTrace(PrintStream s)** This method prints this throwable and its backtrace to the specified print stream.
6	**void printStackTrace(PrintWriter s)** This method prints this throwable and its backtrace to the specified print writer.
7	**void setStackTrace(StackTraceElement[] stackTrace)** This method sets the stack trace elements that will be returned by getStackTrace() and printed by printStackTrace() and related methods.
8	**String toString()** This method returns a short description of this throwable.

Types of Java Exceptions

There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception. However, according to Oracle, there are three types of exceptions namely:

1. Checked Exception
2. Unchecked Exception
3. Error

1) Checked Exception

The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes that inherit the RuntimeException are known as unchecked exceptions. For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

3) Error

Error is irrecoverable. Some example of errors are OutOfMemoryError, VirtualMachineError, AssertionError etc.

Custom exceptions

In Java, we can create our own exceptions that are derived classes of the Exception class. Creating our own Exception is known as custom exception or user-defined exception. Basically, Java custom exceptions are used to customize the exception according to user need.

```
class InvalidAgeException  extends Exception
{
   public InvalidAgeException (String str)
   {
      // calling the constructor of parent Exception
      super(str);
   }
}

// class that uses custom exception InvalidAgeException
public class TestCustomException1
{

   // method to check the age
   static void validate (int age) throws InvalidAgeException{
     if(age < 18){

      // throw an object of user defined exception
      throw new InvalidAgeException("age is not valid to vote");
   }
     else {
      System.out.println("welcome to vote");
      }
    }

   // main method
   public static void main(String args[])
   {
      try
      {
         // calling the method
         validate(13);
      }
      catch (InvalidAgeException ex)
      {
```

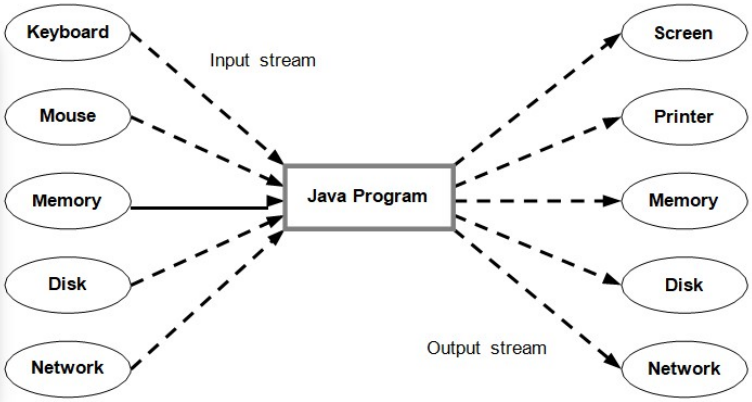
```
        System.out.println("Caught the exception");

        // printing the message from InvalidAgeException object
        System.out.println("Exception occured: " + ex);
    }

    System.out.println("rest of the code...");
  }
}
```

Java I/O and File

Java treats flow of data as stream.

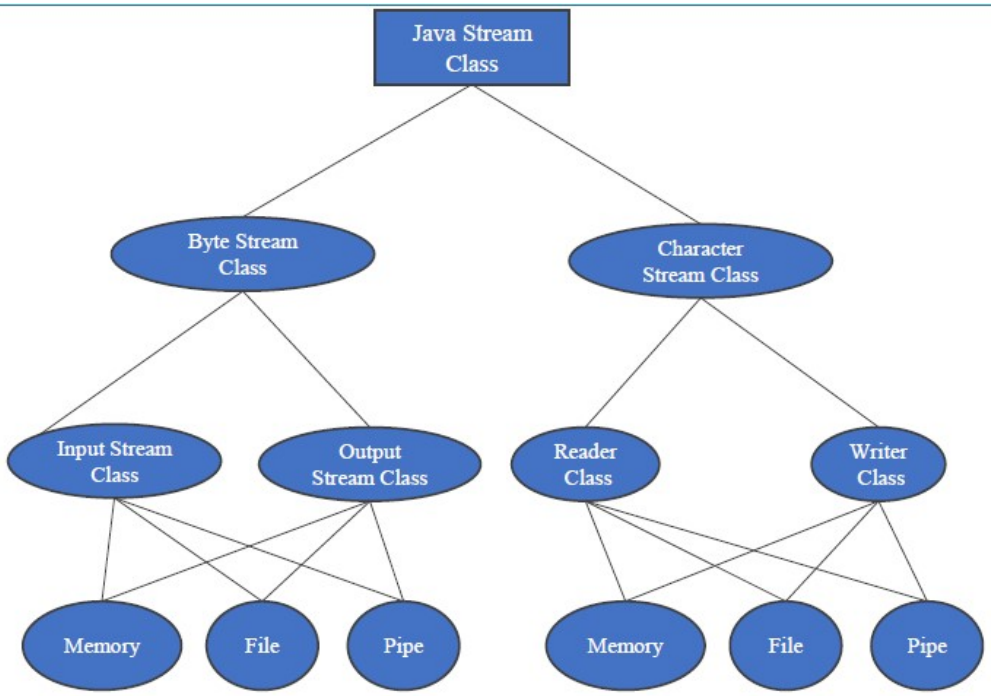
Java streams are classified into two basic types, namely, input stream and output stream.

The java.io package contains a large number of stream classes to support the streams.



Java provides java.io package which contains a large number of stream classes to process all types of data

- Byte stream classes
  - Support for handling I/O operations on bytes
- Character stream classes
  - Supports for handling I/O operations on characters

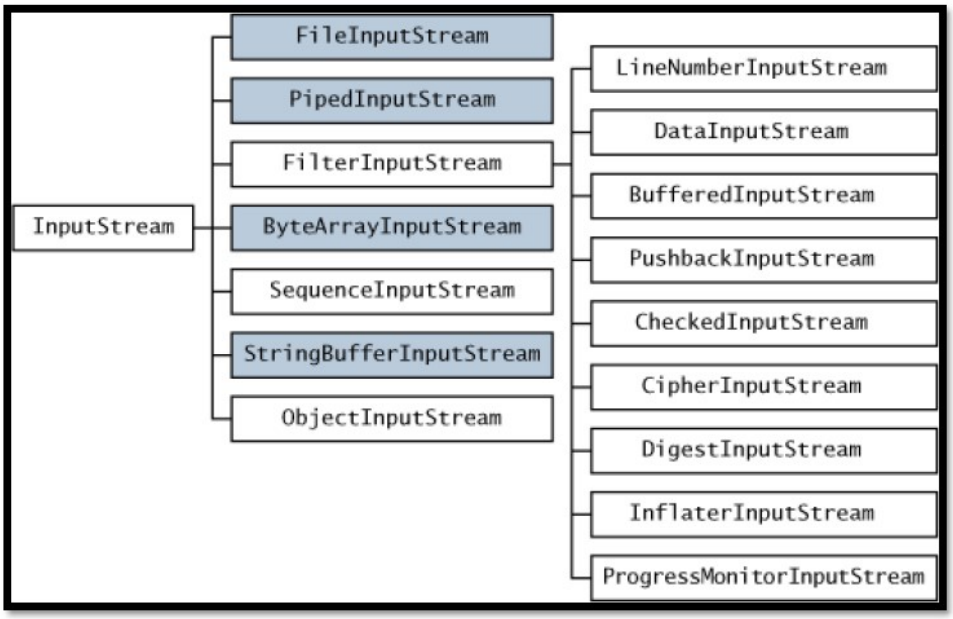


Byte streams

InputStream

InputStream classes are used to read 8-bit bytes and support a number of input-related methods.

Reading bytes
•Closing streams
•Marking positions in streams
•Skipping ahead in streams
•Finding the number of bytes in stream



Methods:

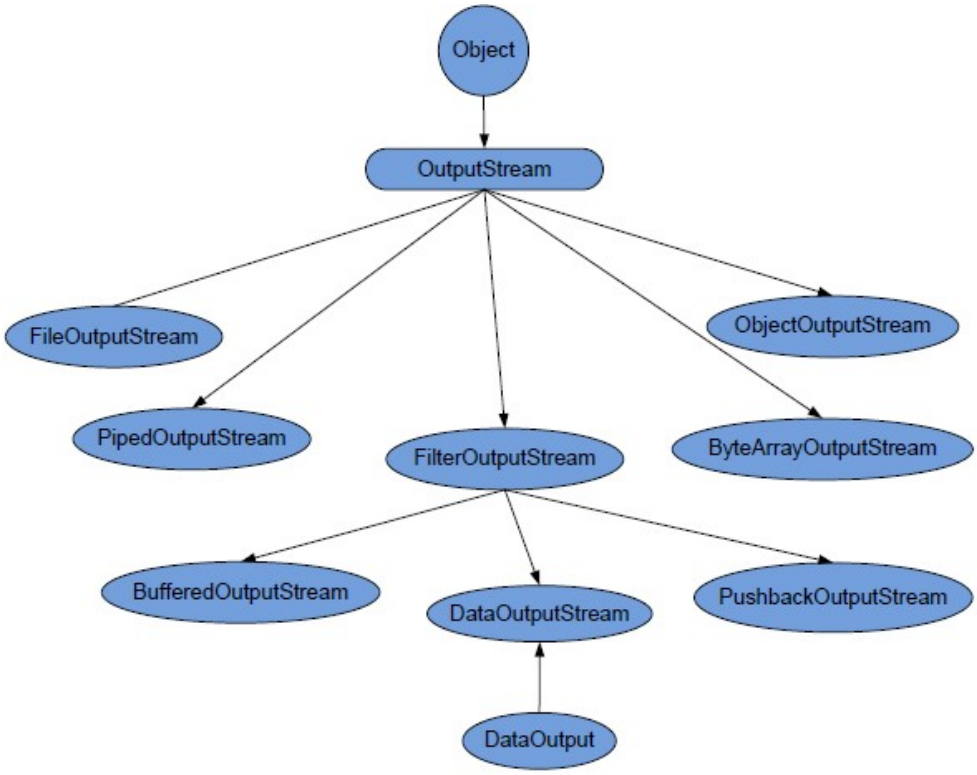
Method	Description
`read( )`	Read a byte from the input stream
`read(byte b[ ])`	Read an array of bytes into b
read(byte b[ ], int n, int m)	Reads m bytes into b starting from n^{th} byte
`available( )`	Gives number of bytes available in the input
`skip(n)`	Skips over n bytes from the input stream
`reset( )`	Goes back to the beginning of the stream
`close( )`	Close the input steam

OutputStream classes is used to write 8-bit bytes and supports a number of input-related methods

Writing bytes
•Closing streams
•Flushing streams

Methods:

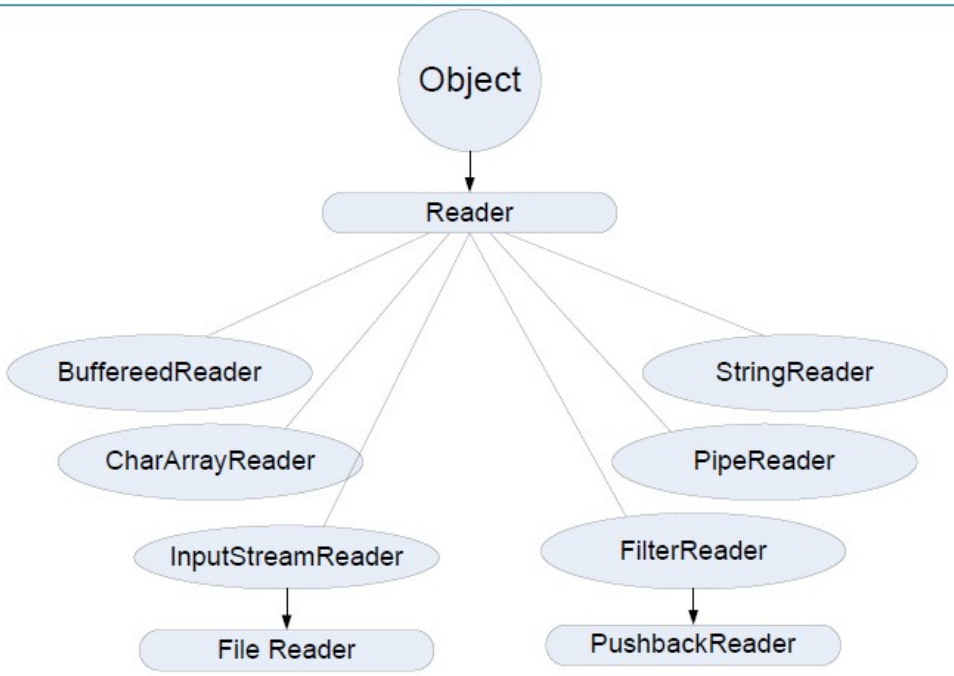
Method	Description
`write ( )`	Write a byte from the input stream
`write (byte b[ ])`	Write all bytes in the array b to the output steam
write (byte b[ ], int n, int m)	Write m bytes from array b starting from n^{th} byte
`close( )`	Close the output stream
`flush( )`	Flushes the output stream

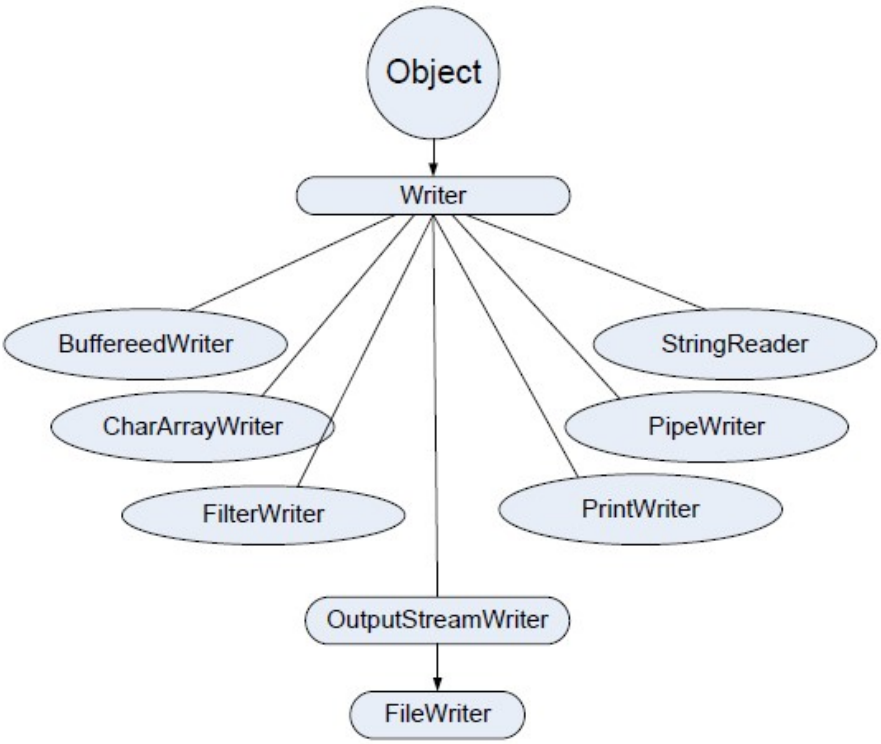


Character Stream Classes

Character stream classes is used to read and write characters and supports a number of input-output related methods

- Reader stream classes
  - To read characters from files.
  - In many way, identical to InputStream classes.
- Writer stream classes
  - To write characters into files.
  - In many way, identical to OutputStream classes.


Object
Reader
BuffereedReader
CharArrayReader
InputStreamReader
File Reader
StringReader
PipeReader
FilterReader
PushbackReader


Object
Writer
BuffereedWriter
CharArrayWriter
FilterWriter
StringReader
PipeWriter
PrintWriter
OutputStreamWriter
FileWriter

Task	Character Stream Class	Byte Stream Class
Performing input operations	Reader	InputStream
Buffering input	BufferedReader	BufferedInputStream
Keeping track of line numbers	LineNumberReader	LineNumberInputStream
Reading from an array	CharArrayReader	ByteArrayInputStream
Translating byte stream into a character stream	InputStreamReader	(none)
Reading from files	FileReader	FileInputStream
Filtering the input	FilterReader	FilterInputStream
Pushing back characters/bytes	PushbackReader	PushbackInputStream
Reading from a pipe	PipedReader	PipedInputStream
Reading from a string	StringReader	StringBufferInputStream
Reading primitive types	(none)	DataInputStream
Performing output operations	Writer	OutputStream
Buffering output	BufferedWriter	BufferedOutputStream
Writing to an array	CharArrayWriter	ByteArrayOutputStream
Filtering the output	FilterWriter	FilterOutputStream
Translating character stream into a byte stream	OutputStreamWriter	(none)
Writing to a file	FileWriter	FileOutputStream
Printing values and objects	PrintWriter	PrintStream
Writing to a pipe	PipedWriter	PipedOutputStream
Writing to a string	StringWriter	(none)
Writing primitive types	(none)	DataOutputStream

Java provides java.io package which includes numerous class definitions and methods to manipulate file and flow of data (called File I/O streams)

There are four major classes:

- File
- FileInputStream
- FileOutputStream
- RandomAccessFile

Opening a File object

- There are three constructors
  - Way 1:
    - `File myFile;`
    - `myFile = new File(fileName);` // Constructor 1
  - Way 2:
    - `File myFile;`
    - `myFile = new File (pathName, filename);` // Constructor 2
  - Way 3:
    - `File myFile;`
    - `File myFile = new File(myDir, fileName);` // Constructor 3

Dealing with file names

- String getName()
- String getPath()
- String getAbsolutePath()
- String getParent()
- boolean renameTo(File newFilename)

Testing a file

- boolean exists()
- boolean canWrite()
- boolean canRead()
- boolean isFile()
- boolean isDirectory()
- boolean isAbsolute()

Getting file information

- `long lastModified()`
- `long length()`
- `boolean delete()`

Directory utilities

- `boolean mkDir(File newDir)`
- `boolean mkDirs(File newDir)`
- `String [] list()`

Reading a File

```
import java.io.*;
class ReadBytes {
      public static void main (String args[])  {
                  FileInputStream infile = null; // Create an input file stream
                  int b;
                  try  {
                       infile = new FileInputStream(args[0]);
                              // Connect infile stream to the required file
                       while((b = infile.read()) != -1)  {
                              System.out.print((char)b); // Read and display data
                       }
                       infile.close();
                  }
                  catch(IOException ioe)  {
                              System.out.println(ioe);
                  }
      }
}
```

Writing into a File

```java
import java.io.*;
class WriteBytes {
      public static void main(String args[]) {
         cities[]={'D','E','L','H','I','\n','M','A','D','R','A','S','\n','L','O','N','D','O',
         'N','\n'}; //Declare and initialize a byte array byte
         FileOutputStream outfile=null; //create an output file stream
         try  {
                 outfile = new FileOutputStream("city.txt");
                  // Connect the outfile stream to "city.txt"
                 outfile.write(cities); //Write data to the stream
                 outfile.close();
         }
         catch(IOException ioe)   {
               System.out.println(ioe);
               System.exit(-1);
         }
      }
}
```

Random Access Files in Java

- As the name implies the class RandomAccessFile allows us to handle a file randomly in contrast to sequentially in InputStream or OutputStream classes
- It allows to move file pointer randomly
- Moreover, it allows read or write or read-write simultaneously.

```java
import java.io.*;
class RandomIO
{
      public static void main (String args[])
      {
         RandomAccessFile file = null;
         try {
              file = new
              RandomAccessFile("rand.dat","rw");
             // Writing to the file
             file.writeChar('X');
            file.writeInt(555);
           file.writeDouble(3.1412);
            file.seek (0);
           // Go to the beginning
           // Reading from the file
          System.out.println(file.readChar());
```

```java
          System.out.println(file.readInt());
          System.out.println(file.readDouble());
          file.seek(2); // Go to the second item
          System.out.println(file.readInt());
          // Go to the end and append false to
          the file
          file.seek(file.length());
          file.writeBoolean(false);
          file. seek (4) ;
          System.out.println(file.readBoolean());
                      file.close();
                      }
                      catch(IOException e)
                      {

          System.out.println(e);
                      }
          }
      }
```